

Inside the Kernel: An Nsight Compute Measurement of Warp Divergence and the FP64 Ceiling

Matias Saavedra

Wednesday 3rd June, 2026

Binary: binary-v5-affinity (fc33e29), HEAD c6bbc8e. Tool: Nsight Compute 2026.2.0.0.
Machine: i7-9750H + GTX 1660 Ti Max-Q (Turing TU116), AC power. Data + capture
script: experiments/16-ncu-divergence/.

Resumen

This report uses NVIDIA Nsight Compute (`ncu`) to measure, for the first time in this project, what happens *inside* the GPU kernel — warp execution efficiency, branch divergence, FP64 pipe utilisation, occupancy, and the compute/memory roofline — across four content regimes (interior, exterior, boundary, canonical), each run GPU-only so the single GPU thread is forced to process *every* region type. The headline: the coherent regions the affinity queue actually routes to the GPU run at **exactly 100.0 % warp execution efficiency and 100.0 % branch efficiency** — zero divergence, now measured rather than argued (`11-priority-queue.tex`). They are **FP64-bound at 85.6 % of the double-precision peak** (~ 110 of 129 GFLOPS at the measured 1.34 GHz on the 1:32 TU116), at 98.9 % occupancy with ~ 0 % memory traffic; the 15 % gap is the dependent iteration recurrence (long-scoreboard stalls, uniform across all regimes), *not* divergence. Divergence is real but **confined to the pixel-floor boundary leaves** (240×135 down to 33.7 %; 480×270 leaves 99.9 %): the uniformity test doubles as a coherence filter. This corrects report 11's ~ 40 % FP64 estimate and closes the divergence question — the GPU lever was never divergence.

1. Why `ncu`, and What It Sees That `nsys` Cannot

Reports 01/04/09/15 profiled with Nsight *Systems* (`nsys`): a *timeline* tracer. It records when each thing runs — CUDA API calls, NVTX ranges, kernel and transfer *durations*, the 12 CPU worker lanes, streams — and is the right tool for every *scheduling* question this project has asked (the `cudaMemcpy` kernel-wait of report 15, the GPU affinity routing of report 12). But `nsys` has *no visibility inside a kernel*. It can say a kernel took 60 ms; it cannot say whether those 60 ms were spent with all 32 lanes of each warp busy or half of them idle, whether the bottleneck was the FP64 pipe or DRAM, or how full the SM was.

Those are *hardware counters*, read only by Nsight *Compute* (`ncu`), which replays each kernel once per counter group. The two tools sit at different altitudes:

Tabla 1: The division of labour. Every prior report used the left column; the divergence question requires the right.

	Nsight Systems (nsys)	Nsight Compute (ncu)
Scope	whole-system timeline	one kernel’s internals
Answers	<i>when</i> / what overlaps	<i>why</i> this kernel is fast/slow
Unit	API calls, NVTX, durations	per-launch HW counters
Overhead	low, one real-time pass	high, replays each kernel
Permissions	user	admin (ERR_NVGPUCTRPERM)

What only **ncu** reaches

(and this report measures): warp execution efficiency (active threads per issued instruction $\nabla \cdot 32$), branch efficiency, FP64 pipe utilisation, achieved occupancy, the Speed-of-Light compute/memory roofline, L1/L2 hit rates, and per-reason warp issue stalls. Divergence, the FP64 ceiling, and the roofline are all *inside* the kernel; **nsys** reports their *symptom* (duration) but never their *cause*.

2. Setup and Methodology

Hardware / tool.

GTX 1660 Ti Max-Q (Turing TU116, 1536 CUDA cores, FP64 at 1:32 of FP32), i7-9750H, AC power. Nsight Compute 2026.2.0.0. Binary **fc33e29** (**binary-v5-affinity**); the kernel is **mandelKernel** with 16×16 blocks, so a 32-thread warp is a 16×2 pixel strip.

GPU-only, by design.

Every regime is run with one worker thread, GPU enabled (**./mandelHybrid spec 1 1 0.1 32768 1 0**). This forces the single GPU thread to process *every* region — interior, exterior, and boundary. In production the affinity queue (report 12) routes boundary regions to the CPU pool, so a normal hybrid run would never show the GPU the divergent regions; to *measure* divergence we must put them on the GPU deliberately.

Four content regimes

(specs in **experiments/16-ncu-divergence/specs/**), chosen to isolate the variables:

Tabla 2: The four regimes and what each forces the GPU to compute.

regime	target	what the GPU sees
interior	inside the main cardioid ($c \approx -0.5$)	six full-frame 1920×1080 leaves, every pixel $\rightarrow$ MAXITER (inset 100%)
exterior	outside the set ($c \approx 1+i$)	six full-frame leaves, every pixel escapes in 1 iteration (inset 0%)
boundary	seahorse valley ($c \approx -0.745+0.113i$)	276 split leaves, 480×270 and 240×135 , mixed escape
canonical	the production spec.in minibrot zoom	300+ leaves, the real mix

Each regime was captured twice: `-set full` (all sections, bounded kernel count) for the roofline/occupancy/stall data, and a cheap two-metric pass for warp + branch efficiency over many more kernels (the distributions of Section 4.3).

Methodology note — the admin gate and a driver skew.

`ncu` counters are admin-restricted (`ERR_NVGPUCTRPERM`), so capture ran from a root script (`capture.sh`) that chowns the reports back; *parsing* a saved `.ncu-rep` needs no privileges (`ncu -import`). A driver / kernel-module version skew (userspace 610.43.02 vs. loaded 595.71.05) disabled `nvidia-smi`/NVML until a reboot reloaded the module; **CUDA and the `ncu` counters were unaffected** (verified: the reports are fully populated). Because Speed-of-Light metrics are clock-normalised ratios, the only casualty was the `gpu_state.csv` snapshot; the GPC clock used below (1.34 GHz) is read from inside the reports.

3. Results

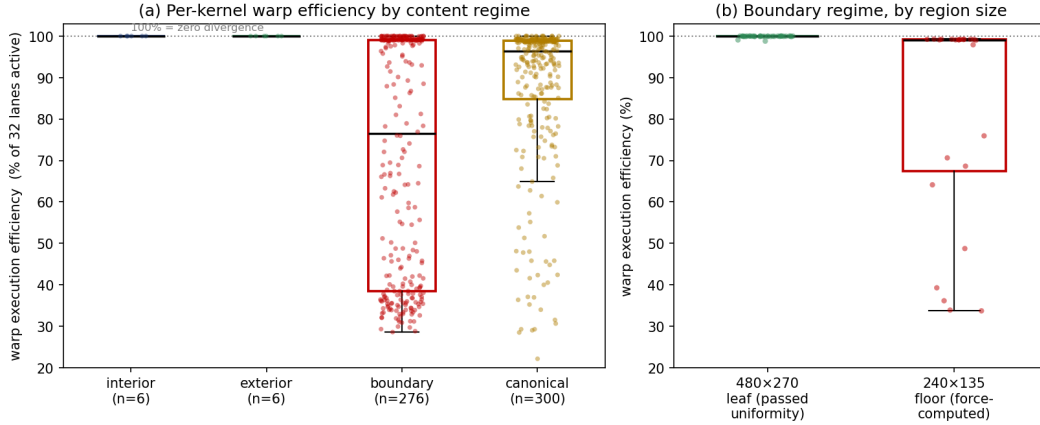


Figure 1: Warp execution efficiency (100 % = all 32 lanes active = zero divergence). **(a)** Per-kernel, by regime: interior and exterior are pinned at 100 %; boundary scatters from 28.6 % to 100 % (median 76.9 %); canonical is mostly high with a divergent tail. **(b)** Within the boundary regime, split by region size: the 480×270 leaves that *passed* the uniformity test are coherent (99.9 %); only the 240×135 *pixel-floor* regions — force-computed when they hit the size limit regardless of coherence — diverge (down to 33.7 %). The split rule is itself a coherence filter.

Tabla 3: Per-regime kernel internals (`–set full`; means over the captured kernels). `warpEff` and `branchEff` are divergence; `FP64` is pipe utilisation; `cSOL`/`mSOL` are the compute/memory Speed-of-Light; `occ` is achieved occupancy. The interior/exterior `warpEff` of *exactly* 100 is the result; the boundary/canonical values here are over the 60 *largest* kernels and so are skewed high — Table 4 gives the honest distribution.

regime	k	warpEff %	branchEff %	FP64 %	cSOL %	mSOL %	occ %	L2 hit %	dur (ms)
interior	6	100.0	100.0	85.6	85.5	0.0	98.9	87.6	600–853
exterior	6	100.0	100.0	85.6	85.4	4.6	94.0	99.4	0.61
boundary	60	92.6*	99.6	85.3	78.1	0.2	84.3	99.2	0.14–71
canonical	60	94.4*	96.4	84.1	69.4	0.9	79.1	94.2	0.01–161

* over the 60 largest kernels (largest-first); see Table 4.

Tabla 4: Warp-efficiency *distribution* over the full kernel population (two-metric pass, `n` kernels each). “<70 %” is the fraction of kernels wasting more than 30 % of their lanes.

regime	n	min	p10	p50	mean	<90 %	<70 %
interior	6	100.0	100.0	100.0	100.0	0 %	0 %
exterior	6	100.0	100.0	100.0	100.0	0 %	0 %
boundary	276	28.6	34.0	76.9	70.7	55 %	47 %
canonical	300	22.2	59.9	96.3	87.6	32 %	13 %

Tabla 5: Boundary regime by region size, and the dominant warp issue stall (mean over the captured kernels). The stall is `long_scoreboard` in *every* regime — see Section 4.2.

regime / size	role	warpEff %	top issue stall
boundary 480×270	leaf (passed uniformity)	99.9 (98.8–100)	—
boundary 240×135	floor (force-computed)	81.6 (33.7–99.3)	—
interior	—	—	<code>long_scoreboard</code> 95 %
exterior	—	—	<code>long_scoreboard</code> 92 %
boundary	—	—	<code>long_scoreboard</code> 95 %
canonical	—	—	<code>long_scoreboard</code> 94 %

4. Analysis

4.1. The coherent regions the GPU actually runs are *exactly* divergence-free

Table 4, Figure 1a: the interior and exterior regimes measure **100.0 % warp execution efficiency** and **100.0 % branch efficiency** on every kernel — all 32 lanes of every warp active on every issued instruction, no divergent branch anywhere.

The mechanism is the escape-time loop and the spatial layout. A warp is a 16×2 pixel strip; the loop bound is data-dependent, so divergence is the *spread* of escape iterations within that strip. In a coherent interior region every pixel runs to **MAXITER** (inset 100 %), so all 32 lanes iterate in lockstep and exit together — spread zero. In a coherent exterior region every pixel escapes at iteration 1 — spread zero again. Report 11 estimated this at “ $\sim 3\%$ of lanes wasted” for a 97 % interior region; at 100 % interior the measurement reaches the limit it predicted: *zero*.

The implication is that the entire “would these boundary regions cause heavy warp divergence on the GPU?” objection (report 11, §3.4) is answered: the regions affinity routes to the GPU are coherent *by construction*, and on them divergence is not small — it is absent.

4.2. The kernel is FP64-bound at 85 % of peak — not 40 % — and the remaining gap is the iteration recurrence, not divergence

Interior: FP64 pipe utilisation 85.6 %, compute Speed-of-Light 85.5 %, at 98.9 % occupancy and $\sim 0\%$ memory throughput (L1 hit 0.6 %). At the measured GPC clock of 1.34 GHz the FP64 peak is $1536/32 \times 2 \times 1.34 = 129$ GFLOPS, so 85.6 % is ~ 110 GFLOPS of double precision. Report 11 *estimated* the coherent region reached “ $\sim 40\%$ of the FP64 ceiling”; the measurement puts it at **85.6 %** — the kernel is far more FP64-saturated than guessed.

The mechanism is in **diverge()**: a serial double-precision recurrence.

Código 1: kernel.cu, **diverge()** – a dependent FP64 chain, data-dependent loop bound.

```

1  __device__ int diverge(double cx, double cy, int MAXITER) {
2  int iter = 0;
3  double vx = cx, vy = cy, tx, ty;
4  while (iter < MAXITER && (vx * vx + vy * vy) < 4) { // bound depends on data
5      tx = vx * vx - vy * vy + cx; // z_{n+1} depends on z_n: a serial
6      ty = 2 * vx * vy + cy;      // double-precision recurrence, no ILP
7      vx = tx;
8      vy = ty;
9      iter++;
10 }
11 return iter;
12 }
```

Two consequences, both measured. First, the loop bound is the only source of divergence — which is why coherent regions (uniform bound) are 100 % efficient. Second, each iterate depends on the previous, so a thread cannot overlap its own FP64 operations; the warp stalls on the latency of the double-precision result. This is exactly what the stall counters show: the dominant issue stall is **long_scoreboard** in *every* regime (92–95 %, Table 5), and it is *uniform across content* — a property of the recurrence, not of the pixels. With the kernel doing essentially no memory loads (mSOL ≈ 0 , L1 hit 0.6 %), that stall is the dependent FP64 chain’s latency, partly — but not fully — hidden by 98.9 % occupancy. The 15 % gap below the FP64 peak is this latency, not occupancy (near full) and not divergence (zero).

The implication is decisive for optimisation: a coherent interior kernel already runs the double-precision pipe at 85 % of a ceiling that is itself 1/32 of FP32. There is no divergence to remove and almost no occupancy or memory headroom. The GPU cannot go faster on this work without doing *fewer iterations* or *wider-than-1:32 arithmetic* — and `float` cannot resolve the deepest frames (report 11), so the pipe rate is a hard floor.

4.3. Divergence is real and severe — but confined to the pixel-floor boundary leaves

The boundary regime is where divergence lives: median warp efficiency 76.9 %, mean 70.7 %, minimum 28.6 % (only 9 of 32 lanes active), and **47 % of boundary kernels waste more than 30 % of their lanes** (Table 4). So the divergent case the GPU is feared for is not a myth — on the boundary it is as bad as 3.5× wasted work.

But Figure 1b and Table 5 localise it exactly. The 480×270 boundary leaves — regions that became leaves because their nine samples *passed* the uniformity test — are 99.9 % efficient. Only the 240×135 *pixel-floor* regions diverge (mean 81.6 %, down to 33.7 %). The reason is the split rule: a region is computed either when it is uniform (low spread) *or* when it falls below `pixelSizeThresh`. The first clause only ever emits *coherent* leaves; the second force-computes whatever remains at the floor, coherent or not. Divergence is therefore quarantined to the floor-clamped leaves — the high-spread regions the uniformity test explicitly refused to compute at full size and split instead. The same `diffThresh` test that drives subdivision (reports 06–09) is, for free, a divergence filter.

The implication is that the GPU never pays a large divergence penalty on a large kernel: the only divergent leaves are the smallest ones.

4.4. In production the GPU’s divergence exposure is near nil

The canonical regime — the production mix, but GPU-only so even the floor regions run on the GPU — has median warp efficiency 96.3 % and only 13 % of kernels below 70 % (Table 4). Two effects compound. First, by §4.3 the divergent leaves are exactly the floor-clamped ones, which are 240×135 = the cheapest tier (sub-millisecond to ~13 ms, vs. the interior leaves’ 600–850 ms). Divergence-weighted by runtime, they are noise. Second — and this run does *not* include it — in real hybrid execution the affinity queue (report 12) routes small regions to the CPU pool, so those divergent floor leaves never reach the GPU at all. The 96.3 % median here is thus a *worst case*; the production GPU sees the 100 % coherent tier almost exclusively.

The implication closes the loop with report 12: GPU affinity is not only a load-balancing win, it is incidentally a *divergence-avoidance* policy — it keeps the GPU on the regions this report measures at 100 % efficiency.

4.5. None of this was visible to nsys

Every number above is a hardware counter. `nsys` (reports 01/09/15) timed these same kernels — it knew the interior kernel took ~700 ms and the boundary kernels 0.1–70 ms — but it could not distinguish a 100%-efficient FP64-bound kernel from a 34%-efficient divergent one, because lane efficiency, FP64 pipe pressure, and the

roofline are *inside* the launch. `15-async-streams-analysis.tex` ranked the GPU near its ceiling from timeline evidence; this report explains *why* from counter evidence — 85 % of a 1:32 FP64 pipe, divergence-free, latency-bound on the recurrence.

5. Recommendations

1. **Do not pursue warp-divergence optimisation for the GPU path.** Measured warp efficiency on the regions affinity routes to the GPU is 100.0 %; there is nothing to recover. Any divergence (down to 34 %) lives in 240×135 floor leaves that are both cheap and CPU-routed. Effort here returns zero.
2. **Treat the FP64 pipe (85.6 % of a 1:32 ceiling) as the hard GPU floor.** The interior kernel is FP64-latency-bound at 98.9 % occupancy with no memory or divergence slack. The only ways down are fewer iterations or wider arithmetic; `float` is precluded by the 15-digit deep-zoom coordinates (report 11). So the GPU per-pixel cost is fixed — consistent with report 15’s “GPU is near its ceiling.”
3. **Periodicity checking must be measured per-executor, not assumed.** The recommended work-reduction lever (skip in-set iterations) is a clean win on the CPU branch, but on the GPU it would convert the currently 100 %-efficient, lockstep interior warps into divergent ones (pixels detecting their cycle at different iterations) *and* add FP64 work per iteration to the bottleneck pipe. Implement it in `compute()`’s CPU branch first; A/B the GPU branch separately against the divergence + FP64 cost this report quantifies.
4. **Keep GPU affinity — it is also a divergence policy.** Routing large coherent regions to the GPU (report 12) keeps it on the 100 %- efficient tier and the divergent floor leaves on the CPU. The two motives (load balance, lane efficiency) point the same way.

6. Conclusion

Nsight Compute converts report 11’s structural argument into measurement. The coherent regions GPU affinity routes to the device run at **exactly 100.0 % warp execution efficiency** — divergence is not small, it is absent — and are **FP64-bound at 85.6 % of the 1:32 double-precision peak** (~ 110 of 129 GFLOPS), at 98.9 % occupancy, ~ 0 % memory, with the remaining gap owed to the serial iteration recurrence (uniform `long_scoreboard` stalls), not divergence. Divergence is real but quarantined to the 240×135 pixel-floor boundary leaves (down to 33.7 %), which are cheap and, in production, CPU-routed — so the uniformity test and the affinity queue together hold the GPU’s divergence exposure near nil. The ~ 40 % FP64 estimate of report 11 is corrected to 85.6 %. The GPU lever was never divergence; it is the iteration count, which only work-reduction can touch.